Session 1: follow along with IBM Bob

Step-by-step instructions and every prompt for today's live activities, from brainstorming a bakery website to your first lap on Corkscrew.

AI Racing League · Learner activity sheet

Session 1 of 4 · Tue 29 Sep 2026 · 3:00 – 4:30 PM GMT · 90 minutes ·

Contents

1. Before the session
2. Using IBM Bob: the basics
3. Part 1 · Bob at work
4. A1 · Brainstorm
5. A2 · Gather requirements
6. A3 · Plan
7. A4 · Build
8. A5 · Improve and troubleshoot
9. A6 · Document and create content
10. A7 · Automate, and explain an error
11. A8 · Your turn: your own job
12. Part 2 · The race car
13. R1 · Car on the grid and baseline lap
14. R2 · Bob explains the car's code
15. R3 · Pick your first upgrade idea
16. Lap log
17. Before Session 2
18. Words you'll hear
19. Links

Follow along with your coach, or work through this sheet on your own after the session. Every prompt has a **Copy prompt** button. You don't need any coding experience.

Before the session

☐ VS Code installed, with IBM Bob installed and signed in (ibm.biz/IBMBoBUniversity).

☐ Python 3 installed. On Windows, tick **Add Python to PATH** during install.

☐ TORCS and the Python scripts downloaded and unzipped to a simple folder with no spaces, e.g. `C:\RaceYourCode`.

☐ An empty folder called `bob-at-work` on your desktop.

☐ `Menu.html` saved into `bob-at-work`: open the lab link, click **Raw**, then File → Save As.

☐ Your mandatory learning plan activities started on SkillsBuild.

> Can't install something in time? Come anyway. You can watch today and catch up with this sheet afterwards.

Using IBM Bob: the basics

Mode	Use it to	Can it change files?
Ask	Ask questions, get explanations, brainstorm	No
Plan	Turn a goal into a step-by-step plan	No code changes
Code	Create and edit files	Yes, after you approve

- Switch modes with the selector at the bottom of the Bob chat.
- Type `@` then a file name to point Bob at a file, e.g. `@index.html`.
- Stay in the **same chat** when an activity builds on the last one. Start a **new task** for a new project.
- Keep **auto-approval off**. Read each request from Bob before clicking Approve.
- A good prompt has a **scope** (which file), a **goal**, a **shape** (table, list, steps) and **guardrails** ("don't edit yet").

Part 1 · Bob at work

> **The scenario:** You're opening Sprout and Crumb, a small vegan bakery. You need a simple website and some help running the business, but you can't code. IBM Bob is your partner.

A1 · Brainstorm

3 min · Ask · Idea and discovery

Goal: Get a wide range of ideas for what a bakery website could do, and pick the ones you like.

Steps

1. Open VS Code, then File → Open Folder and choose an empty folder called `bob-at-work`.
2. Open the IBM Bob panel and pick **Ask** in the mode selector at the bottom of the chat.

3. Copy the prompt below into Bob and press Enter.
4. Read the table and circle three ideas you like.

Prompts

Prompt · Ask mode

```
I'm opening a small vegan bakery called Sprout and Crumb. I can't code.
Brainstorm 8 things a website could do for a bakery like mine.
Put them in a table with the columns: idea, who it helps, effort (low / medium
/ high).
Keep the language simple.
```

If you get stuck

- **Can't find the Bob panel:** Click the Bob icon in the left activity bar, or open the Command Palette (Ctrl/Cmd+Shift+P) and search for Bob. If it's missing, the extension or Bob IDE isn't installed yet: they watch today and install from ibm.biz/IBMBoBUniversity afterwards.
- **Bob asks them to sign in, or says usage is used up:** They need to sign in with their IBMid. On the free tier, remind everyone to keep prompts focused and to avoid re-running the same prompt.
- **Learner is in Code mode and Bob tries to create a file:** Tell them to click Reject, switch to Ask and re-send. Good moment to repeat: the mode decides what Bob is allowed to do.

My notes

What idea did Bob suggest that you wouldn't have thought of?

_____

_____

_____

A2 · Gather requirements

3 min · Ask · Requirements and planning

Goal: Turn three chosen ideas into a prioritised list of requirements, and answer Bob's questions.

Steps

1. Stay in the **same chat** so Bob remembers the brainstorm, and stay in **Ask** mode.
2. Send the first prompt below.
3. Bob will ask you some questions. Reply with the second prompt, or with your own answers.
4. Check the must-have list. Does it match what you'd want as a bakery owner?

Prompts

Prompt · Ask mode

```
From your list, I want: a menu, our story, and a preorder form.
Turn these into clear requirements for the website.
```

```
Group them as must have, should have, nice to have.
For the preorder form, list every field and what makes an entry valid.
Ask me up to 3 questions about anything that's unclear.
```

Prompt · Follow-up · Ask mode

```
Pickup only, Tuesday to Saturday. Orders need 48 hours' notice. No payment
online.
```

If you get stuck

- **Bob doesn't remember the brainstorm:** They started a new task. Either scroll back to the earlier chat or paste the three ideas into the prompt.
- **Bob doesn't ask any questions:** Fine. Ask them to reply: "What else would you need to know before building this?"
- **Learners think requirements are "too technical":** Reframe: requirements are just a list of what it must do. Bob wrote them in plain English on request.

My notes

Which of Bob's questions surprised you? What would have gone wrong if nobody asked it?

_____

_____

_____

A3 · Plan

4 min · Plan · Requirements and planning

Goal: Get a build plan for a one-page site, then make it simpler with one follow-up.

Steps

1. Switch the mode selector to **Plan**. Stay in the same chat.
2. Send the first prompt.
3. Read the plan. Then send the second prompt to simplify it.
4. If Bob offers to save the plan as a file, either choice is fine.

Prompts

Prompt · Plan mode

```
Using these requirements, plan a simple one-page website for Sprout and Crumb.
Give me a step-by-step build plan: the page sections in order, what goes in
each, and how the preorder form should behave.
It must be one HTML file that works on phones. Don't build it yet.
```

Prompt · Follow-up · Plan mode

```
Make it simpler: remove anything that needs a server or a database, and explain
the plan for someone who isn't a developer.
```

If you get stuck

- **Can't find Plan mode:** Some installs name or group modes differently. Use Ask mode with the same prompt: the output is the same idea.
- **Bob starts writing code in the plan:** Remind it: "Plan only, no code yet."
- **The plan mentions servers, databases or hosting:** That's what the second prompt is for. Explain: a form that sends real orders needs a server; ours only shows a thank-you message.

My notes

What did you change in the plan, and why is that easier now than after building?

_____

_____

_____

A4 · Build

5 min · Code · Build

Goal: Have Bob build the bakery page, open it in your browser, then change it with one follow-up.

Steps

1. Switch to **Code** mode. Stay in the same chat.
2. Send the first prompt. Bob will ask permission to create `index.html`: read the request, then click **Approve**.
3. Open the page: right-click `index.html` in the file list → Reveal in File Explorer (Windows) or Reveal in Finder (Mac) → double-click the file.
4. Try the form with an empty field, then filled in.
5. Send the second prompt, approve the change, and refresh the browser.

Prompts

Prompt · Code mode

```
Build the plan as a single file called index.html with the CSS and JavaScript
inside it.
Use 6 placeholder menu items with prices in pounds.
The preorder form should check the fields are filled in and show a thank-you
message. It doesn't need to send anything.
Add short comments so a beginner can follow the code.
```

Prompt · Follow-up · Code mode

```
Change the colours to soft greens and add a "Today's special" banner at the
top.
```

If you get stuck

- **Bob can't create the file or asks where to put it:** No folder is open in VS Code. File → Open Folder → bob-at-work, then send the prompt again.
- **Nothing happens after sending:** Look for an approval button waiting in the Bob panel. It's easy to miss.
- **index.html opens as code, not a web page:** It opened inside VS Code. Use Reveal in File Explorer/Finder and open it from there in a browser.

My notes

How long would this page have taken you a year ago? What did you still have to decide yourself?

_____

_____

_____

A5 · Improve and troubleshoot

5 min · Code · Validate and improve

Goal: Find out why the Roast Toast Coast Café page is broken, then let Bob fix it and review the changes.

> **Before you start:** Download `Menu.html` from the lab's GitHub page using the **Raw** button, then File → Save As into `bob-at-work`.

Steps

1. Make sure `Menu.html` is in your `bob-at-work` folder and open it in VS Code.
2. Start a **new task** in Bob, check you're in **Code** mode, and send the first prompt. Approve when Bob asks to read the file.
3. Read the findings. Find one syntax issue and one functional issue.
4. Send the second prompt. Review the side-by-side view before you approve.
5. Compare: which change makes the **Our Menu** button work?

Prompts

Prompt · Code mode

```
Can you identify all syntax and functional errors in this HTML file:
@Menu.html?
Perform a thorough analysis.
Group the findings by severity (critical, high, medium, low).
Indicate whether each issue is related to syntax, functionality, or structure.
Identify anything that could prevent the page from rendering or behaving
correctly.
```

Prompt · Follow-up · Code mode

```
Rewrite this file to fix the identified issues.
Apply the recommendations from your previous analysis.
Preserve the original structure where possible.
```

If you get stuck

- **Menu.html is full of GitHub page code:** They saved the GitHub web page, not the file. Use the Raw button first, then Save As.
- **The image still doesn't show after the fix:** The path is now correct, but the image file itself isn't in their folder. That's expected; the lab doesn't render the page. The menu button fix is the proof.
- **@Menu.html doesn't autocomplete:** The file must be inside the open folder. Or type the full file path, as in the lab.

My notes

Which issue would have been hardest to spot by eye? Why did grouping by severity help?

_____

_____

_____

A6 · Document and create content

3 min · Code · Document and deliver

Goal: Get a plain-English guide for the site and a launch post plus talk outline, then check them for accuracy.

Steps

1. Go back to the Sprout and Crumb chat (or start a new task and use `@index.html`). Stay in **Code** mode.
2. Send the prompt and approve the two new files.
3. Open `README.md` and press Ctrl+Shift+V (Windows) or Cmd+Shift+V (Mac) to see the formatted preview.
4. Check one instruction in the README against the real page. Is it true?

Prompts

Prompt · Code mode

```
Create README.md for the Sprout and Crumb site in @index.html, written for the
bakery owner, not a developer.
Explain what each section does and how to change a menu item or price.
Then create launch-post.md with a short Instagram caption and a 5-slide outline
for a talk to local cafés: a title, 3 bullet points and a visual idea for each
slide.
```

If you get stuck

- **README describes a feature the page doesn't have:** Great teaching moment. Ask Bob: "Check README.md against index.html and correct anything that isn't true."
- **Markdown looks messy:** They're looking at the raw file. Use the preview shortcut.
- **Bob creates only one file:** Ask for the second file in a follow-up.

My notes

Who else in your job would benefit from documentation you currently don't have time to write?

_____

_____

_____

A7 · Automate, and explain an error

5 min · Code, then Ask · Build + improve

Goal: Have Bob write a script that summarises weekly preorders, run it yourself, then break it on purpose and ask Bob to explain the error.

Steps

1. In **Code** mode, send the first prompt and approve the two new files.
2. Open a terminal: Terminal → New Terminal. It opens in your folder.
3. Type `python summarise_orders.py` (on Mac: `python3 summarise_orders.py`) and press Enter. You just ran Python!
4. Break it on purpose: rename `preorders.csv` to `orders-old.csv` and run the command again.
5. Copy the red error, switch Bob to **Ask**, and send the second prompt with the error pasted in.
6. Rename the file back to `preorders.csv` and run it once more.

Prompts

Prompt · Code mode

```
Automate a weekly job for me.
1. Create preorders.csv with 15 made-up preorders: name, item, quantity, pickup
date.
2. Write summarise_orders.py that reads the file and prints the total quantity
per item and the number of orders per pickup date, sorted.
Use only Python's built-in libraries. Add beginner-friendly comments. Tell me
the exact command to run it.
```

Prompt · Follow-up · Ask mode

```
I ran summarise_orders.py and got this error:
[paste the error]
Explain it in plain English and tell me how to fix it. Don't change any files.
```

If you get stuck

- **'python' is not recognised, or the Microsoft Store opens:** Python isn't installed or wasn't added to PATH. Reinstall with "Add Python to PATH" ticked, then close and reopen VS Code. They'll need this for the race anyway.
- **Mac says 'command not found: python':** Use python3.
- **"No such file or directory: summarise_orders.py":** The terminal is in a different folder. Run cd followed by the folder path, or close the terminal and open a new one with the folder open in VS Code.

Want more?

Ask Bob in Code mode: "Turn our launch into launch-checklist.md with tasks, owners and deadlines for the next two weeks."

My notes

What weekly task in your job follows the same steps every time? That's your automation candidate.

_____

_____

_____

A8 · Your turn: your own job

3 min · Ask or Plan · Any stage

Goal: Write one prompt about a real task from your job and get three ideas from Bob.

Steps

1. Start a new task in **Ask** mode.
2. Fill in the two brackets with your own role and task, then send.
3. Post your favourite idea in the session chat.

Prompts

Prompt · Ask mode

```
I work as [your role]. I regularly have to [a task that takes you time].
Suggest three ways you could help with it, from simplest to most ambitious.
For the simplest one, show me the exact prompt I should use.
```

If you get stuck

- **Learner can't think of a task:** Suggest: summarising meeting notes, drafting a weekly update, turning notes into slides, building a checklist.
- **Learner has no Bob access:** Write the prompt anyway and try it after the session.

My notes

Transition: "Now let's use the same Bob, the same prompts, on something with four wheels."

_____

_____

_____

Part 2 · The race car

Your mission: the fastest single lap of Corkscrew from a standing start, with **zero damage**. A Python script drives the car; you and Bob improve the script. Today you only watch and understand it. You start changing it in Session 2.

R1 · Car on the grid and baseline lap

12 min · None (TORCS + terminal) · Discovery

Goal: Get the unmodified car round Corkscrew once and record the time to beat.

Steps

1. Start TORCS (double-click `wtorcs.exe` in your TORCS folder). If Windows warns you, click More info → Run anyway.
2. Race → Quick Race → Configure Race.
3. Track: Road Tracks → Corkscrew → Accept.
4. Drivers: make sure only `scr_server 1` is in the Selected list → Accept.
5. Laps: 1 → Accept → New Race. A blue waiting screen means TORCS is ready.
6. In a terminal, go to the gym_torcs folder: `cd` followed by the folder path (e.g. `cd C:\RaceYourCode\gym_torcs`).
7. Run `python snakeoil3_gym.py` (Mac/Linux: `python3`). Watch the car drive.
8. When the lap ends, read Time, Damage and Top Speed from the panel in the top-left and write them in your lap log.

If you get stuck

- **Windows SmartScreen blocks TORCS:** More info → Run anyway.
- **Firewall pop-up when running Python or TORCS:** Allow access on private networks. Port 3001 is how Python talks to TORCS.
- **Script waits forever or says connection refused:** TORCS must be on the blue waiting screen first. Check the driver is scr_server 1. If a previous run was stopped, start a New Race again.

My notes

Everyone who finished posts baseline time and damage in chat.

_____

_____

_____

R2 · Bob explains the car's code

8 min · Ask · Discovery

Goal: Understand, in plain English, how the Python script drives the car and why it picks up damage.

Steps

1. In VS Code, File → Open Folder → your `gym_torcs` folder.
2. Start a new task in Bob, choose **Ask**, and send the first prompt. Approve when Bob asks to read the file.
3. Click one of the line numbers Bob gives you and find that line in the code.
4. Send the second prompt with that line number.

Prompts

Prompt · Ask mode

```
I've never coded before. Explain how @snakeoil3_gym.py drives the race car.
1. In plain English, what does the car check and decide each moment?
2. Which values could I change to make it faster or safer? Give line numbers.
3. Why does it pick up damage in corners?
Do not change any code.
```

Prompt · Follow-up · Ask mode

```
Explain line [number] like I've never coded. What would happen on track if I
changed it?
```

If you get stuck

- **Bob analyses the whole library, not the driving part:** Follow up: "Focus only on the function that makes the driving decisions."
- **Bob proposes code changes:** Decline for today. Changes start in Session 2 with a copy of the file.
- **Line numbers don't match:** Bob may count differently or the file differs. Ask Bob to quote the line instead of numbering it.

My notes

What's one thing the car does that a human driver wouldn't?

_____

_____

_____

R3 · Pick your first upgrade idea

8 min · Ask (optional) · Planning

Goal: Agree one upgrade to build in Session 2 and write it in your lap log.

Steps

1. Form a team, or decide to work solo.
2. Look at Bob's answer. Optionally send the prompt below.
3. Write your chosen idea in the v1 row of your lap log. Don't build it yet.
4. Post your team name and idea in chat.

Prompts

Prompt · Ask mode

```
Of the reasons you gave for the car being slow or damaged, which single change
is the safest to try first for a beginner? Explain the expected effect and the
risk.
```

If you get stuck

- **Learner wants to change everything:** One change per test, so you know what moved the lap time. That's the core rule from Session 2 onwards.
- **Confident coders want to build now:** Fine as homework, in a copy of the file, with the lap log updated.

My notes

Preview Session 2: "On Thursday, Bob builds your idea and we race it."

_____

_____

_____

Lap log

Read Time, Damage and Top Speed from the top-left panel in TORCS at the end of each lap. This log is your evidence for the final submission.

Version	What changed	Lap time	Damage	Top speed	Keep?
base	Nothing, original script				Yes
v1	My idea for Session 2:				
v2					
v3					

Before Session 2 ·

☐ Complete your mandatory learning plan activities on SkillsBuild. These are individual, even if you race as a team.

☐ Get TORCS running and record your baseline lap, if you didn't today.

☐ Try one Bob prompt for your own job and keep the result.

☐ Choose your team (or go solo) and write your first upgrade idea in the lap log.

☐ Post your baseline time on Discord.

Words you'll hear

Prompt	What you type to Bob: your request in plain English
VS Code	A free editor for code files, like Word for programs
Python	The programming language that drives the race car
Script	A file of instructions, e.g. `summarise_orders.py`
Terminal	A window where you type commands to run things
Diff	A before-and-after view of what Bob changed
CSV	A simple spreadsheet saved as text, one row per line
TORCS	The Open Racing Car Simulator, the game our code drives
Syntax / logical / functional error	Breaks the language rules / runs but gives the wrong result / something it should do is missing

Links

- Get IBM Bob: https://ibm.biz/IBMBoBUniversity
- Learning plan (SkillsBuild): https://skills.yourlearning.ibm.com/activity/PLAN-6FA860C53B3A
- TORCS and Python scripts: https://ibm.box.com/s/sud4g3o1mqyfqoa4ab9lr80vh8zc1ri6
- Café lab file: Menu.html (use the Raw button): https://github.com/academic-initiative/skillsbuild/blob/main/ibm-bob/troubleshoot-your-code/file/Menu.html
- Windows setup video: https://youtu.be/rRbpOUH0lFQ
- macOS extra steps (Wine): https://ibm.box.com/v/MacOSExtraExt